Assessment II - Case Study / Mini Project (40 Marks)

Academic Year : 2026 - 2027
Class : III CSE (Sections A & B)
Course Code & Name : U21AMP05 & Computer Vision
Course Coordinator : Mr. Pandiya Rajan G, Assistant Professor III, AM
Course Instructor : Mr. G. M. Ganesh, Assistant Professor I, CSE

How this assessment works

- Work in the team of 4 given in the table below. Each team has one project. Every member must contribute code, results and part of the report.
- Use your own experiments. Libraries and tutorials are fine, but copying a complete notebook from Kaggle or GitHub and submitting it is not. Every figure and number in your report must come from code you ran.
- Cite every dataset, paper and code source you use. Details on report format, datasets and submission are on the later pages.
- Each member will be asked questions on the whole project in a short viva. Individual marks may be adjusted based on the viva.

S. No.	Title of the Assessment II	Roll Nos.
1	**Title: Smart Document Scanner and Enhancer** (Units I, II) Dataset: 60+ phone photos of notebook pages and printed sheets (self-captured) plus the SmartDoc dataset. Build a scanner that finds the page, corrects the perspective and cleans up lighting and shadows. Measure how much the cleaned image improves OCR accuracy.	24CS002, 24CS003, 24CS008, 24CS017
2	**Title: Chest X-ray Enhancement and Its Effect on Pneumonia Classification** (Units II, V) Dataset: Chest X-Ray Images (Pneumonia), Kaggle. Compare histogram equalization, contrast stretching and CLAHE on X-rays. Train the same classifier on original and enhanced images and see whether enhancement actually helps.	24CS024, 24CS025, 24CS026, 24CS027
3	**Title: Image Denoising Benchmark: Spatial vs Frequency Filters** (Unit II) Dataset: BSDS500 / Set12 clean images with noise added by you. Compare mean, median, Gaussian, bilateral, non-local means and FFT low-pass filters. Find which filter suits which noise type, using PSNR and SSIM.	24CS028, 24CS032, 24CS035, 24CS037
4	**Title: Lane Detection for Road Videos** (Unit III) Dataset: TuSimple lane detection frames plus your own dashcam/phone video. Use Canny, Hough transform and perspective warp to detect and overlay lanes. Measure accuracy against labeled frames and study failure cases.	24CS041, 24CS042, 24CS052, 24CS056
5	**Title: Brain MRI Tumor Segmentation: Classical Methods vs U-Net** (Units III, V) Dataset: LGG MRI Segmentation (Kaggle) with ground-truth masks. Build a threshold/watershed baseline and a U-Net. Compare using Dice and IoU.	24CS073, 24CS074, 24CS076, 24CS085
6	**Title: Safety Helmet Detection using YOLO** (Units III, V) Dataset: hard hat / safety helmet datasets from Roboflow Universe or Kaggle. Train a YOLO detector and compare with a second detector. Report mAP and FPS, and run it on a live video.	24CS090, 24CS091, 24CS095, 24CS096

S. No.	Title of the Assessment II	Roll Nos.
7	**Title: Panorama Stitching and Feature Detector Comparison** (Unit IV) Dataset: own overlapping photo sets plus the Oxford Affine Covariant Regions dataset. Compare SIFT, ORB and AKAZE with Brute-force and FLANN matching. Stitch panoramas using RANSAC homography.	24CS099, 24CS103, 24CS104, 24CS106
8	**Title: Face Recognition Attendance System** (Units III, IV) Dataset: consented face photos collected in class, plus LFW for unknown faces. Haar cascade detection with LBPH and HoG+SVM recognition. Log attendance to a CSV file and measure error rates.	24CS115, 24CS119, 24CS124, 24CS253
9	**Title: Vehicle Counting and Tracking in Traffic Video** (Units III, IV) Dataset: traffic videos (UA-DETRAC samples, free stock clips, or your own). Background subtraction plus optical flow to detect and track vehicles and count them at a line. Compare counts with manual counting and with a YOLO-based count.	24CS258, 24CS260, 25CSL02, 25CSL07
10	**Title: Plant Leaf Disease Classification with Transfer Learning** (Unit V) Dataset: PlantVillage (Kaggle / TensorFlow Datasets). Compare a CNN from scratch with MobileNetV2 or ResNet50 transfer learning. Use Grad-CAM and your own phone photos to test if the model really works.	25CSL08, 25CSL09, 25CSL10, 25CSL11

Detailed Project Briefs

Each brief says what to build, where to get data, what steps to follow and what must appear in the report. Dataset names are given so you can search for them; links and folder structures change, so use the official page or the Kaggle/Roboflow search. Check the license of any dataset you download.

Project 1: Smart Document Scanner and Enhancer (Units I, II)

Team: 24CS002, 24CS003, 24CS008, 24CS017

Problem	Photos of notes and forms taken on a phone are tilted, shadowed and unevenly lit. The aim is a pipeline that turns such a photo into a flat, clean, readable scan.
Dataset and where to find it	• **Self-captured:** at least 60 photos (notebook pages, printed sheets, a board) under room light, window light and with a hand shadow, at different angles. • **SmartDoc (ICDAR 2015 challenge)** for a public test set with known page corners. Search 'SmartDoc dataset ICDAR 2015'. • **Kaggle** (search 'document images', 'scanned documents') for extra samples.
What to do	1. Read the image, convert to grayscale, blur and run Canny edge detection. 2. Find the largest 4-point contour and apply a perspective transform (getPerspectiveTransform). 3. Enhance with three methods: global threshold, adaptive threshold and CLAHE. Remove shadows with background estimation (morphological closing, then divide). 4. Run Tesseract (pytesseract) before and after enhancement and compute character error rate against typed ground-truth text for at least 15 pages. 5. Wrap everything in a small command-line tool or Streamlit page.
What must appear in the report	Block diagram, before/after image grid, % of images where corners were found correctly, OCR error rate table (original vs each method), 5 failure cases with reasons.

Project 2: Chest X-ray Enhancement and Its Effect on Pneumonia Classification (Units II, V)

Team: 24CS024, 24CS025, 24CS026, 24CS027

Problem	X-ray images often have low contrast. Enhancement looks better to the eye, but does it help a classifier? Test this properly instead of assuming.
Dataset and where to find it	• **Chest X-Ray Images (Pneumonia)** on Kaggle (uploaded by Paul Mooney, originally from Kermany et al.). About 5,800 images, two classes. Use a 1,500 to 2,000 image subset if Colab is slow. • Note: the original validation folder is tiny. Make your own train/validation/test split (70/15/15).
What to do	1. Check class balance and image sizes; resize to 224x224. 2. Apply histogram equalization, contrast stretching and CLAHE (try clip limits 2, 4, 8 and tile sizes 8x8, 16x16). Compare histograms and entropy. 3. Train one fixed model (small CNN or fine-tuned ResNet18) separately on: original, HE, stretching, CLAHE. Keep every other setting identical and use the same seed. 4. Report accuracy, precision, recall, F1 and confusion matrix for each. Repeat with 3 seeds and give mean and spread. 5. Discuss: does better looking mean better accuracy? Which errors (false negatives) matter most here?
What must appear in the report	Histogram plots, enhancement comparison figure, results table over 4 variants and 3 seeds. Add a line stating this is a classroom study and not a diagnostic tool.

Project 3: Image Denoising Benchmark: Spatial vs Frequency Filters (Unit II)

Team: 24CS028, 24CS032, 24CS035, 24CS037

Problem	Different noise needs different filters. Build a benchmark that shows, with numbers, which filter works best for Gaussian, salt-and-pepper and speckle noise.
Dataset and where to find it	• **BSDS500** (Berkeley Segmentation Dataset) from the UC Berkeley vision group page; use 30 to 50 images. • **Set12 / BSD68** are small standard denoising test sets; search the names on GitHub or Kaggle. • One or two real noisy photos (low-light, high ISO) taken on your phone for a visual check.
What to do	1. Convert to grayscale and add Gaussian (sigma 10, 25, 50), salt-and-pepper (2%, 5%, 10%) and speckle noise. 2. Implement/apply: mean, median, Gaussian, bilateral, non-local means (cv2.fastNlMeansDenoising), and an ideal and Butterworth low-pass filter in the frequency domain (NumPy FFT). 3. Tune each filter on a few images (kernel size, sigma, cutoff frequency) with a small grid search, then fix the best values for the full test. 4. Compute PSNR and SSIM against the clean image, plus run time per image. 5. Plot the FFT spectrum of clean, noisy and filtered images and explain what the filter removed.
What must appear in the report	Table of PSNR/SSIM per noise type and level, runtime chart, best-filter recommendation per noise type, zoomed crops showing edge blur or ringing.

Project 4: Lane Detection for Road Videos (Unit III)

Team: 24CS041, 24CS042, 24CS052, 24CS056

Problem	Detect lane lines in a driving video using classical edge and line detection, then check how reliable it is when lighting and road conditions change.
Dataset and where to find it	• **TuSimple lane detection benchmark** (frames with labeled lane points); mirrors are available on Kaggle. Search 'TuSimple lane detection'. • **Own video:** record 2 to 3 minutes from a vehicle or a bike on a road near campus (mounted phone, daytime and dusk if possible). • Free dashcam clips from Pexels or Pixabay for extra variety.
What to do	1. Mask a region of interest; threshold white and yellow lane colors in HLS color space. 2. Edge detection with Canny, then line detection with the Hough transform. 3. Warp to a bird's-eye view and fit lane curves (sliding window plus second-order polynomial). Estimate curvature and vehicle offset from center. 4. Smooth over the last few frames so the lane does not jump when one frame fails. 5. Evaluate on at least 200 labeled TuSimple frames (a lane point is correct if within a fixed pixel tolerance) and report accuracy and false positives.
What must appear in the report	Output video with overlay, accuracy numbers, and a failure analysis for shadows, worn paint, curves, night and rain.

Project 5: Brain MRI Tumor Segmentation: Classical Methods vs U-Net (Units III, V)

Team: 24CS073, 24CS074, 24CS076, 24CS085

Problem	Mark the tumor region in brain MRI slices. First see how far classical segmentation gets, then see what a U-Net adds.
Dataset and where to find it	• **LGG MRI Segmentation** on Kaggle (Mateusz Buda), from The Cancer Imaging Archive. About 110 patients, slices with tumor masks.
What to do	1. Look at the data; keep slices with and without tumor and note the imbalance. 2. Split by patient, not by slice, so the test patients are never seen in training. 3. Baseline: Otsu or adaptive threshold, morphological opening/closing, then watershed or region growing. 4. Train a small U-Net (PyTorch or Keras, 128x128 or 256x256 input) with Dice plus BCE loss and simple augmentation (flip, rotate). 5. Compute Dice and IoU on the test patients for both methods; show overlays of good, average and bad cases.
What must appear in the report	Segmentation overlays, Dice/IoU table, training curves, discussion of why the classical method fails on some slices. Classroom study only.

Project 6: Safety Helmet Detection using YOLO (Units III, V)

Team: 24CS090, 24CS091, 24CS095, 24CS096

Problem	Detect whether people at a site or on a two-wheeler are wearing helmets. Train a custom object detector and judge it on both accuracy and speed.
Dataset and where to find it	• **Roboflow Universe** (universe.roboflow.com): search 'hard hat' or 'safety helmet'; export in YOLO format. • **Kaggle:** search 'Hard Hat Detection' or 'Helmet Detection'. • Optional: 100 of your own photos labeled with LabelImg or Roboflow to test on unseen images.
What to do	1. Check class names, count boxes per class and fix obviously wrong labels. 2. Train YOLOv8n with the Ultralytics library on Colab (GPU). Try 2 settings of image size or epochs. 3. Train or run a second detector (YOLOv8s, or SSD MobileNet) on the same split. 4. Report precision, recall and mAP@0.5 per class, plus FPS on CPU and GPU. 5. Run on a webcam or recorded video; collect misses (small, occluded, night, side view) and explain them.
What must appear in the report	Training curves, PR curve, confusion matrix, detector comparison table (accuracy vs speed), a short demo video.

Project 7: Panorama Stitching and Feature Detector Comparison (Unit IV)

Team: 24CS099, 24CS103, 24CS104, 24CS106

Problem	Join overlapping photos into one panorama, and find out which feature detector and matcher gives the best result for the effort.
Dataset and where to find it	• **Own photos:** three sets (building, room interior, open landscape), 3 to 5 photos each with 20 to 30% overlap, taken by rotating on the spot. • **Oxford Affine Covariant Regions dataset** (VGG, University of Oxford) has image sequences (graf, bark, bikes) with ground-truth homographies for scoring. • OpenCV sample images (github.com/opencv/opencv, samples/data folder) for quick tests.
What to do	1. Detect and describe keypoints with SIFT, ORB and AKAZE. (SURF is patented and available only in the opencv-contrib non-free build, so use AKAZE instead.) 2. Match with brute-force and FLANN, filtering with Lowe's ratio test (try 0.6, 0.7, 0.8). 3. Estimate homography with RANSAC, warp and blend the images into a panorama. 4. On the Oxford sequences compare estimated and ground-truth homography (reprojection error). Record keypoints found, good matches, inlier ratio and time. 5. Show a case where stitching fails (low overlap, repeated texture, moving people) and explain why.
What must appear in the report	Panorama results, comparison table (keypoints, matches, inliers, error, time) for 3 detectors x 2 matchers, discussion on speed vs robustness.

Project 8: Face Recognition Attendance System (Units III, IV)

Team: 24CS115, 24CS119, 24CS124, 24CS253

Problem	Detect and recognize faces from a camera and mark attendance automatically using hand-crafted features, and measure how often it gets people wrong.
Dataset and where to find it	• **Own data:** 10 to 15 volunteer classmates, at least 20 images each with different lighting, angle and expression. Take written or verbal consent, keep the images only for this project and delete them after evaluation. • **Labeled Faces in the Wild (LFW)** from the UMass vision lab, used as 'unknown people' to test false accepts. scikit-learn can also download it (fetch_lfw_people). • Olivetti faces (in scikit-learn) for a quick first test.
What to do	1. Detect faces with a Haar cascade; crop, convert to grayscale, resize and equalize the histogram. 2. Method 1: LBPH recognizer (cv2.face, needs opencv-contrib-python). Method 2: HoG features with an SVM. 3. Choose a confidence threshold so that unknown faces are rejected rather than forced into a name. 4. Evaluate on held-out images: accuracy, false accept rate on LFW faces, and effect of dim light and side angles. 5. Write name and time to a CSV file when a person is recognized; avoid duplicate entries within a session. Test whether a printed photo fools the system and note it.
What must appear in the report	Accuracy and false-accept table for both methods, threshold plot, sample attendance CSV, short discussion on privacy and spoofing.

Project 9: Vehicle Counting and Tracking in Traffic Video (Units III, IV)

Team: 24CS258, 24CS260, 25CSL02, 25CSL07

Problem	Count vehicles crossing a road using motion cues, and check how close classical methods come to a manual count and a deep detector.
Dataset and where to find it	• **UA-DETRAC** (traffic sequences with annotations); search the name and use a few sample sequences. • **Own video:** 3 clips of 1 to 2 minutes filmed from a footbridge or a high point, camera fixed on a tripod or wall. • Free traffic footage on Pexels or Pixabay.
What to do	1. Subtract background with MOG2 or KNN; clean the mask with morphology; extract contours above a minimum area. 2. Estimate motion using Lucas-Kanade (sparse) or Farneback (dense) optical flow and plot flow vectors for a few frames. 3. Track each vehicle with a simple centroid tracker and count it when it crosses a virtual line, one count per direction. 4. Manually count vehicles in all clips (keep your sheet) and compute the percentage count error. 5. Run YOLOv8n on the same clips and compare counts, speed and failure cases (shadows, merged vehicles, stopped vehicles).
What must appear in the report	Output videos with counts, error table for classical vs YOLO on 3 clips, analysis of where background subtraction fails.

Project 10: Plant Leaf Disease Classification with Transfer Learning (Unit V)

Team: 25CSL08, 25CSL09, 25CSL10, 25CSL11

Problem	Identify crop leaf diseases from a photo. Show how much transfer learning helps and test whether a model trained on lab images works on real leaf photos.
Dataset and where to find it	• **PlantVillage** on Kaggle (search 'PlantVillage dataset') or in TensorFlow Datasets as 'plant_village'. Choose one crop such as tomato (about 10 classes) to keep training time reasonable. • **PlantDoc** (images from real fields) as a harder test set. Search 'PlantDoc dataset'. • 20 to 30 photos of real leaves taken by you for a final check.
What to do	1. Explore class counts; split into train/val/test before any augmentation; apply augmentation only to training data. 2. Baseline: a small CNN trained from scratch. 3. Transfer learning: MobileNetV2 or ResNet50 pretrained on ImageNet, first frozen then fine-tuned. 4. Report accuracy, per-class F1 and confusion matrix; compare training time and model size. 5. Use Grad-CAM to check the model looks at leaf spots, not the background. Test on your own photos and report the accuracy drop.
What must appear in the report	Comparison table (scratch vs transfer), Grad-CAM figures, honest note on the gap between lab images and real photos.

Common Guidelines for All Teams

1. Tools you can use

- Python 3.10+, OpenCV (opencv-python; opencv-contrib-python where noted), NumPy, Matplotlib, scikit-image, scikit-learn.
- Deep learning: PyTorch or TensorFlow/Keras. Ultralytics for YOLO.
- Free GPU: Google Colab or Kaggle Notebooks. Use Git for version control (one shared repository per team).

2. Where to look for datasets

Source	What it is good for
Kaggle Datasets (kaggle.com/datasets)	Medical, plant, traffic and general classification/segmentation datasets. Free account needed.
Roboflow Universe (universe.roboflow.com)	Ready-made object detection datasets already labeled; export in YOLO format.
Hugging Face Datasets (huggingface.co/datasets)	Many vision datasets, loadable with the datasets library.
TensorFlow Datasets and torchvision.datasets	Standard datasets that download with one line of code (for example PlantVillage in TensorFlow Datasets, CIFAR and GTSRB in torchvision).
The Cancer Imaging Archive (TCIA)	Medical imaging (MRI, CT). Check the usage terms.
University dataset pages	BSDS500 (UC Berkeley), Oxford VGG, LFW (UMass), UA-DETRAC and similar benchmark sets.
Papers with Code (paperswithcode.com/datasets)	Find which benchmark datasets researchers use for a task, with links.
Your own data	Phone photos and videos. Take consent when people appear in them.

3. Experiment rules

- Always split train, validation and test data first. Never tune on the test set.
- Fix a random seed and write it in the report. For trained models, run at least 2 to 3 seeds where time allows.
- Compare at least two methods or settings. A single result with no comparison will score low on Results and Analysis.
- Include failure cases. Showing where your method breaks is part of the analysis.
- Medical and face data are sensitive. Use them for learning only, do not upload personal images to public repositories.

4. Report format (12 to 20 pages, PDF)

- Title page: project title, team roll numbers, course details.
- Problem statement and objectives; why it matters in computer vision.
- Dataset: source, size, classes, split, preprocessing.
- Methodology: block diagram of the pipeline and a short explanation of each block.
- Implementation: tools, important parameters, key code snippets (full code goes in the repository).
- Results: tables, graphs and sample outputs, with interpretation and comparison.
- Analysis, limitations, conclusion and future work.
- References (IEEE style) and a short table of who did what.

5. What to submit

- Report PDF.
- Code: GitHub repository link or zip with README (how to run), requirements.txt and, if small, sample data. Do not upload the full dataset.
- Demo: live run or a 2 to 3 minute screen recording, followed by the viva. Submission and viva dates will be announced by the course instructor.

Course Instructor **Course Coordinator** **HoD / CSE**